Bisección y dividir y vencerás

Programación: encontrar la función y plantear la recursión

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada

3 Dividir y vencerás: plantear la recursión

4 Problemas para practicar

Contenido

1. Bisección: encontrar la función
   - Concepto
   - Ejemplo
   - Ejercicio
2. Bisección: cuando la función ya viene dada
3. Dividir y vencerás: plantear la recursión
4. Problemas para practicar

El problema que ya vieron

El punto 1 del parcial

En una calle hay N casas, cada una en el número que es su distancia desde el comienzo. Se instalan M antenas, en cualquier punto. Se pide la menor distancia máxima posible entre una casa y su antena más cercana.

Lo primero que se intenta

Probar dónde poner las antenas. Las posiciones son números reales, así que no hay nada que recorrer: el espacio de respuestas es infinito.

Cambiar la pregunta

De pedir el óptimo a comprobar un candidato

Encontrar el mejor valor es difícil. Comprobar si un valor dado sirve es mucho más fácil, y con eso basta:

$$\varphi(d) : \text{¿alcanzan } M \text{ antenas si cada una cubre hasta } d \text{ metros?}$$

Por qué basta

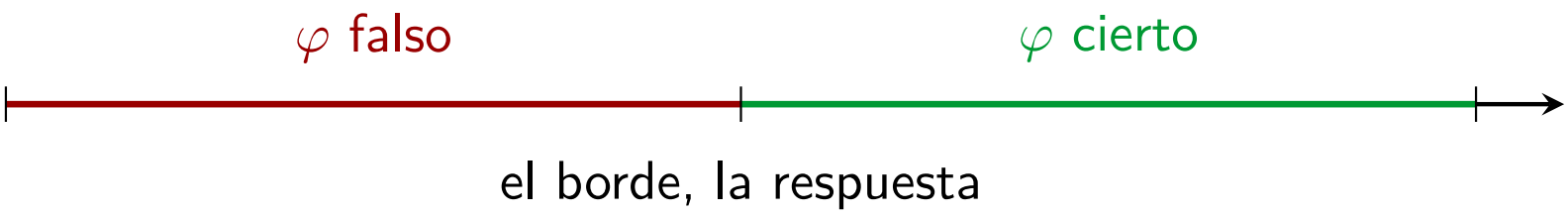
Si se sabe contestar φ, la respuesta del problema es el menor d donde φ pasa de falso a cierto. Buscar ese borde es lo que hace la bisección.

Cinco pistas para dar con la función

Qué mirar en el enunciado

1. La respuesta es un número, y piden el menor o el mayor que cumple algo.
2. Ese algo se comprueba para un candidato fijo en una pasada.
3. Al comprobarlo, el candidato pasa de incógnita a dato.
4. Si un candidato sirve, los de un lado también.
5. La respuesta está acotada por el peor y el mejor caso.

La monotonía es lo que hay que verificar



Qué se está diciendo

Si con d alcanzan M antenas, con cualquier $d' > d$ también, porque cada antena cubre más. Los candidatos quedan partidos en dos tramos y hay un solo borde.

Sin monotonía no hay bisección

Mirar el punto medio solo sirve si de ahí se deduce en qué mitad está la respuesta. Esa deducción es la monotonía, y por eso se comprueba antes de escribir el ciclo.

El ejemplo del enunciado

Cuatro casas y dos antenas

Casas en 12, 21, 7 y 24, con $M = 2$. El enunciado dice que la respuesta es 2,5, con las antenas en 9,5 y en 22.

Cómo se comprueba un candidato

Con las casas ordenadas, se toma la primera sin cubrir y se pone su antena lo más a la derecha posible, en casa $+ d$, que cubre hasta casa $+ 2d$. Se repite y se cuentan las antenas.

Dos candidatos, paso a paso

d	primera casa libre	antena en	cubre hasta
2	7	9	11
	12	14	16
	21	23	25

Tres antenas

Con $d = 2$ hacen falta tres y solo hay dos: $\varphi(2)$ es falso.

Dos candidatos, paso a paso

d	primera casa libre	antena en	cubre hasta
2,5	7	9,5	12
	21	23,5	26

Dos antenas

Con $d = 2{,}5$ bastan dos: $\varphi(2{,}5)$ es cierto. El borde está entre 2 y 2,5, y la bisección lo encuentra.

Los límites y las vueltas

Dónde buscar

Por debajo, $d = 0$: cada casa necesitaría su antena. Por arriba, la longitud de la calle: una sola antena cubre todo. La respuesta está en ese intervalo.

Cuándo parar

La respuesta es real, así que no hay un entero al que llegar. Se parte el intervalo a la mitad cien veces, que lo dejan en 10^{-30} del tamaño original. Cada vuelta recorre las casas una vez.

UVa 12032, The Monkey and the Oiled Bamboo

El problema

Una escalera tiene n peldaños a alturas $r_1 < \cdots < r_n$. Se sube de uno en uno, sin saltarse ninguno. Se arranca con fuerza k: ningún salto puede medir más de k, y el salto que mide exactamente k baja la fuerza en uno. Se pide el menor k con el que se llega arriba.

Antes de programar, conteste

¿Cuál es $\varphi(k)$? ¿Por qué es monótona? ¿Entre qué dos valores está la respuesta?

Enunciado y envío: `https://vjudge.net/problem/UVA-12032`

La lectura de la entrada

```
int casos;
scanf("%d", &casos);
for (int caso = 1; caso <= casos; caso++) {
  int n;
  scanf("%d", &n);
  vector<int> alturas(n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &alturas[i]);
  }
  /* aquí va la bisección sobre k */
```

Por qué sirve así

Las alturas vienen en una sola línea, pero `scanf` con `%d` se salta espacios y saltos de línea por igual, de modo que el mismo ciclo las lee sin importar cómo estén repartidas.

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada
- Concepto
- Ejercicio

3 Dividir y vencerás: plantear la recursión

4 Problemas para practicar

El otro caso

A veces no hay que inventar nada

Hay enunciados que entregan la función escrita y piden el valor donde vale cero. No hay que convertir optimización en decisión, pero quedan dos cosas por comprobar, y son las mismas de antes.

Las dos preguntas

1. ¿Es monótona en el intervalo donde se busca?
2. ¿Los extremos del intervalo tienen signos distintos?

La monotonía se comprueba derivando

Un caso chico

$f(x) = -3x + 1$ en $[0, 1]$. Se tiene $f'(x) = -3 < 0$, así que f decrece. Además $f(0) = 1 > 0$ y $f(1) = -2 < 0$, de modo que cruza el cero una sola vez.

Qué se hace con eso

Si f decrece, el punto medio positivo deja la raíz a la derecha y el negativo a la izquierda. Esa es la regla que mueve los extremos.

Cuando no hay raíz

El error que cuesta el problema

Si se bisecta sin mirar los extremos, el ciclo igual termina y devuelve un número, que no es raíz de nada. El resultado se ve razonable y está mal.

La comprobación

Con f decreciente, hay raíz en $[a, b]$ exactamente cuando $f(a) \geq 0 \geq f(b)$. Si $f(a) < 0$ la función ya empieza por debajo, y si $f(b) > 0$ nunca alcanza a bajar.

UVa 10341, Solve It

El problema

Dados seis enteros con $0 \leq p, r \leq 20$ y $-20 \leq q, s, t \leq 0$, se busca x en $[0, 1]$ con

$$pe^{-x} + q \sin x + r \cos x + s \tan x + tx^2 + u = 0,$$

con cuatro decimales, o `No solution` si no hay.

Antes de programar, conteste

¿Por qué los rangos dados garantizan que la función no crece? ¿Qué dos comprobaciones van antes del ciclo?

Enunciado y envío: `https://vjudge.net/problem/UVA-10341`

La lectura de la entrada

```c
int p, q, r, s, t, u;
while (scanf("%d %d %d %d %d %d", &p, &q, &r, &s, &t, &u) == 6) {
  /* un caso por línea, hasta el fin del archivo */
}
```

El fin del archivo

El enunciado no dice cuántos casos hay. `scanf` devuelve cuántos valores logró leer, así que compararlo contra seis es la forma de saber cuándo se acabó la entrada.

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada

3 Dividir y vencerás: plantear la recursión
- Concepto
- Ejemplo
- Ejercicio

4 Problemas para practicar

Otra pregunta

Contar parejas fuera de orden

En una lista de enteros distintos, una pareja está fuera de orden cuando el de la izquierda es mayor que el de la derecha. ¿Cuántas hay?

Lo directo

Dos ciclos anidados, comparando cada pareja: $\Theta(n^2)$. Con medio millón de elementos son 10^{11} comparaciones y no termina.

La plantilla

Tres pasos sobre un tramo $[ini, fin)$

1. **Dividir**: partir el tramo por la mitad, en $mid = (ini + fin)/2$.
2. **Conquistar**: resolver los dos tramos con la misma función.
3. **Combinar**: armar la respuesta del tramo con las dos respuestas y lo que falte calcular.

Por índices, no copiando

La función recibe *ini* y *fin* y trabaja sobre el mismo arreglo. Partir copiando mitades cuesta $\Theta(n)$ por nivel y cambia el costo total.

De dónde sale la recursión

Las parejas del tramo son de tres clases

1. Las dos puntas en la mitad izquierda.
2. Las dos puntas en la mitad derecha.
3. Una punta en cada mitad.

Lo que eso da

Las dos primeras clases son el mismo problema sobre la mitad del tamaño, y ahí está la recursión. La tercera es lo que hay que saber combinar.

El caso base

Un tramo de un elemento no tiene ninguna pareja, y por eso la recursión para ahí.

La combinación sale gratis si las mitades quedan ordenadas

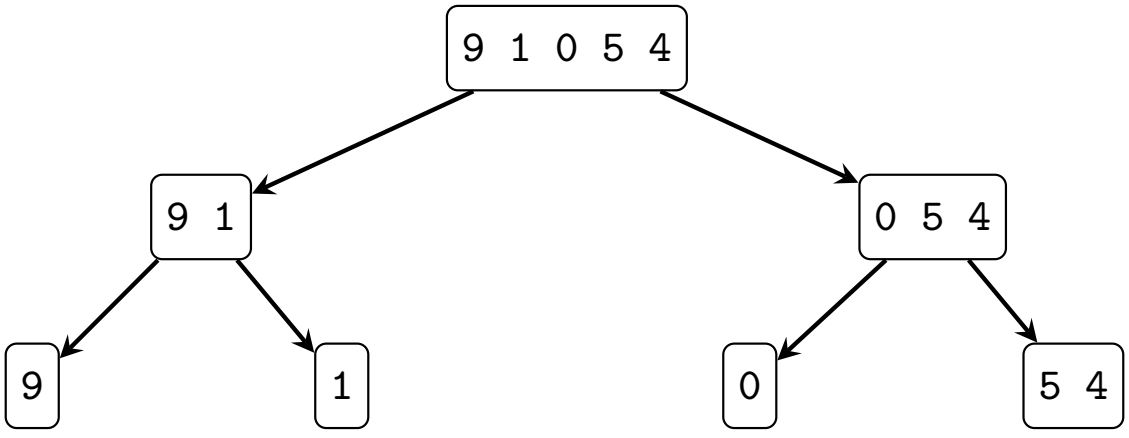
La observación

Al mezclar dos mitades ya ordenadas, cuando sale un elemento de la derecha, todos los que siguen pendientes en la izquierda son mayores que él. Esa salida aporta tantas parejas como elementos le falten a la izquierda.

Por qué conviene ordenar

Ordenar no es un extra que se agrega: es lo que permite contar la tercera clase en una sola pasada en vez de comparar todo contra todo.

La lista del enunciado, por mitades



Qué devuelve cada llamada

Cada una entrega dos cosas: cuántas parejas fuera de orden tenía su tramo y el tramo ya ordenado, que es lo que la llamada de arriba va a mezclar.

Las cuentas de abajo hacia arriba

Tramo	Al mezclar	Parejas	Queda
9 y 1	sale 1 y faltaba 9	1	1 9
5 y 4	sale 4 y faltaba 5	1	4 5
0 y 4 5	sale 0, nada pendiente	0	0 4 5

Las tres de abajo

Cada una mezcla dos tramos chicos y cuenta lo que cruza.

Las cuentas de abajo hacia arriba

Tramo	Al mezclar	Parejas	Queda
1 9 y 0 4 5	sale 0 y faltan 1 y 9; sale 4 y falta 9; sale 5 y falta 9	4	0 1 4 5 9

El total

$1 + 1 + 0 + 4 = 6$, que es lo que responde el enunciado para 9 1 0 5 4.

Lo que cuesta

La recurrencia

Cada llamada resuelve dos tramos de la mitad y después recorre el tramo completo una vez para mezclar:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

Cómo se lee

Hay $\log_2 n$ niveles y cada nivel mueve los n elementos una vez, así que el total es $\Theta(n \log n)$. Contra los $\Theta(n^2)$ del principio, con medio millón de elementos son nueve millones de operaciones en vez de cien mil millones.

UVa 10810, Ultra-QuickSort

El problema

Dada una lista de $n < 500\,000$ enteros distintos, se pide cuántos intercambios de elementos vecinos hacen falta para dejarla ordenada de menor a mayor. Varios casos; un caso con $n = 0$ cierra la entrada.

Antes de programar, conteste

¿Por qué el número de intercambios de vecinos es exactamente el número de parejas fuera de orden? ¿De qué tamaño es la respuesta en el peor caso?

Enunciado y envío: `https://vjudge.net/problem/UVA-10810`

La lectura de la entrada

```
int n;
while (scanf("%d", &n) == 1 && n != 0) {
  vector<int> a(n), aux(n);
  for (int i = 0; i < n; i++) {
    scanf("%d", &a[i]);
  }
  printf("%lld\n", contar(a, aux, 0, n));
```

Dos detalles

El auxiliar de la mezcla se reserva una vez por caso y se reutiliza en todas las llamadas. Y el contador va en `long long`: en el peor caso la respuesta es del orden de 10^{11} y no cabe en 32 bits.

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada

3 Dividir y vencerás: plantear la recursión

4 Problemas para practicar

Para seguir

Bisección con otro disfraz

UVa 714, repartir libros entre escribas para que el que más copie copie lo menos posible. **UVa 11413**, repartir barriles entre contenedores minimizando el mayor. En los dos la pregunta es *¿alcanza con t?*

Recursión y estructuras

UVa 839, un móvil de barras colgadas de barras, donde el árbol de llamadas es el móvil. **UVa 10935** y **UVa 12100**, dos colas. **UVa 156**, agrupar palabras por sus letras ordenadas.